

AppEntrada Logística — Informe para sistemas

Repositorio: <https://github.com/fer788/appentrada-logistica.git>

Rama principal: master

Nota: documento vivo; revisar último commit en GitHub.

1. Propósito

Aplicación web interna de **calendario de entregas / logística**: grilla semanal (lunes a sábado), viajes por casilla, ABM de clientes, productos y choferes, exportación de datos, marcado de entregas realizadas, navegación por semanas del año, etc.

2. Arquitectura técnica

Capa	Tecnología	Ubicación en el repo
Frontend	React 18, Vite 6	src/, build en dist/
Backend API	Node.js, Express	server/index.js
Base de datos	MySQL 8.x	Esquema y migraciones en server/db.js

Flujo lógico (desarrollo típico)

1. El navegador carga la interfaz desde **Vite** (puerto **5173**).
2. Las peticiones a /api/* las recibe Vite y las **proxifica** al backend Express (**4000**).
3. Express persiste y consulta datos en **MySQL** (**3306**).

```
[Navegador] → http://<host>:5173 → Vite (proxy /api)
                                   → Express :4000 → MySQL :3306
```

En **producción** sobre una misma VM, el mismo proceso Node puede servir la API y los archivos estáticos generados en dist/ (ver sección 7).

3. Ubicación del código (referencia actual)

Descripción	Ruta típica
Proyecto en disco de desarrollo	C:\AiProjects\AppEntrada\logistica
Copia en estructura local aiprojects	C:\aiprojects\AppEntrada\dev\logistica

El **origen de verdad** del código versionado es el repositorio Git remoto indicado arriba.

4. Configuración y secretos

- Archivo **.env** en la raíz del proyecto (no se sube al repositorio).

- Plantilla de referencia: **.env.example**.

Variable	Uso
PORT	Puerto del servidor Express (por defecto 4000)
MYSQL_HOST	Host de MySQL (ej. 127.0.0.1 o hostname de la VM)
MYSQL_PORT	Puerto MySQL (típicamente 3306)
MYSQL_USER / MYSQL_PASSWORD	Credenciales
MYSQL_DATABASE	Nombre de la base (por defecto logistica_entregas)

Recomendación: en producción usar usuario MySQL dedicado con permisos mínimos sobre esa base, no root sin contraseña.

5. Base de datos

- **Motor:** MySQL 8.x.
- **Base lógica:** `logistica_entregas` (configurable).
- **Tablas principales:** `clientes`, `choferes`, `productos`, `viajes` (esquema evolutivo vía migraciones al arrancar el servidor en `server/db.js`).

Datos en disco (modo desarrollo Windows)

Si se usa `npm run mysql:start`, puede utilizarse un **datadir local** del proyecto: carpeta `logistica\.mysql-data\`.

En un **servidor o VM corporativa** lo habitual es un **servicio MySQL** con datadir estándar; el proyecto solo requiere la conexión vía `.env`.

6. Puertos y acceso en red

Servicio	Puerto	Notas
Vite (solo desarrollo)	5173	UI; con <code>host: true</code> acepta conexiones LAN
Express API	4000	API REST; en dev el navegador suele ir solo a 5173 (proxy)
MySQL	3306	No exponer a internet; en LAN solo si hay requisito explícito

Acceso desde otras PCs en la misma red (modo desarrollo): `http://<IPv4-de1-servidor>:5173`

El firewall del equipo servidor debe permitir **TCP 5173** (perfil **Privada**). Script de ayuda: `npm run lan:firewall` (PowerShell como administrador).

7. Comandos operativos

Comando	Descripción
<code>npm install</code>	Dependencias
<code>npm run mysql:start</code>	Intenta levantar MySQL local (Windows; ver <code>scripts/start-mysql.js</code>)
<code>npm run dev</code>	API + frontend en modo desarrollo
<code>npm run build</code>	Genera <code>dist/</code> para producción
<code>npm start</code>	Servidor Node en producción (API + estáticos desde <code>dist/</code>)

8. Automatización en la estación actual (referencia)

- Acceso directo en el **Inicio de sesión** de Windows que lanza MySQL + `npm run dev`.
- Tarea programada tipo **KeepAlive** que ejecuta periódicamente `scripts/ensure-stack.ps1`.

En la **VM de producción** conviene un **servicio** (Windows Service, PM2, NSSM, systemd, etc.) y supervisión estándar.

9. Respaldo y recuperación

Qué	Recomendación
Base de datos	<code>mysqldump</code> periódico de <code>logistica_entregas</code> + retención
Código	GitHub + procedimientos internos
<code>.env</code>	Respaldo cifrado o gestor de secretos; no en el repo
<code>.mysql-data</code> (si aplica)	Respaldo en frío con MySQL detenido, o dumps lógicos

Recuperación típica: restaurar dump SQL, desplegar commit acordado, colocar `.env`, arrancar `npm start` o servicio equivalente.

10. Migración recomendada a máquina virtual (VM)

1. **MySQL** como servicio en la VM; BD y usuario con permisos acotados.
2. **Node.js LTS** en la VM.
3. Clonar el repositorio, configurar `.env` apuntando al MySQL de la VM.
4. `npm ci` → `npm run build` → `npm start` (o servicio gestionado).
5. Opcional: **proxy inverso** (IIS/nginx) con **HTTPS** y firewall mínimo.
6. MySQL solo en **localhost** de la VM salvo necesidad explícita.

11. Seguridad (resumen)

- No exponer MySQL a internet sin túnel/VPN y hardening.
- En producción, evitar Vite; usar `build` + `npm start` o contenedor.

- Revisar firewall y segmentación con sistemas.

12. Contacto y mantenimiento

- **Código e issues:** repositorio GitHub del proyecto.
- **Releases:** definir política interna (tags, changelog).

Archivos en docs/

Archivo	Uso
INFRA-SISTEMAS.md	Fuente editable (Markdown)
INFRA-SISTEMAS.html	Navegador o Imprimir → Guardar como PDF
INFRA-SISTEMAS.pdf	PDF para compartir (<code>npm run docs:pdf</code>)

Documento para el área de sistemas. Versión editable: docs/INFRA-SISTEMAS.md en el repositorio.